# Assignment 3

**Notes**

Where appropriate, I have taken the output from code cells and embedded them into the metadata of this file. This is so the results of the file are usable without having to wait for things to render. If you wish to see the computationally-intensive methods working, I would suggest trying them with a very low fidelity (for instance, a low `num_alphas` in `bifurcation_diagram` ).

Versions:

- Python: 3.13.5
- NumPy: 2.4.2
- SciPy: 1.14.0
- SciPy: 1.17.1
- Pandas: 3.0.2
- Matplotlib: 3.10.9

In [1]:
```
import numpy as np
import sympy as sp
import scipy as sc
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import scipy.integrate as sc_int
from matplotlib.animation import FuncAnimation
from IPython.display import HTML, display
```

# 1. Introduction

In this document, we study the motion of a pendulum by comparing theoretical predictions to real world data.

The angle $\theta(t)$ (radians) at which a pendulum hangs is governed by the following ODE:

$$\ddot{\theta} + 2\gamma\dot{\theta} + \omega^2 \sin(\theta) = 0,$$

where $\gamma \geq 0$ is a "damping coefficient" (how much friction slows the pendulum down) and $\omega > 0$ is the natural frequency of the pendulum. $\omega$ is related to the length of the pendulum $\ell$ and the acceleration incurred on it by gravity: $\omega = \sqrt{g/\ell}$. Note that $\theta$ is a function of time.

The above equation has no closed-form expression, so we often consider the system only when $\theta$ is small. In this case, we can apply the approximation $\sin(\theta) \approx \theta$ to simplify the equation to $\ddot{\theta} + 2\gamma\dot{\theta} + \omega^2\theta = 0$. There is indeed a closed-form expression to this linearised model, as we are now working with a second-order ODE with constant coefficients.

$$\theta(t) = \theta_0 c(t) + \left(\frac{\eta_0 + \gamma\theta_0}{\omega_d}\right) s(t)$$

is that solution when we are considering the *underdamped* condition ($\gamma < \omega$) and initial conditions $\theta(0) = \theta_0$ and $\dot{\theta}(0) = \eta_0$. The functions $c(t)$, $s(t)$ and constant $\omega_d$ are defined as follows:

$$c(t) = e^{-\gamma t}\cos(\omega_d t)$$
$$s(t) = e^{-\gamma t}\sin(\omega_d t)$$
$$\omega_d = \sqrt{\omega^2 - \gamma^2}$$

$\omega_d$ is the *damped natural frequency*.

We will now proceed to analyse this system numerically, by investigating how well it fits real-world data, and how different parameter values effect the behaviour of the pendulum's motion.

# 2. Verifying the exact solution

### 2.1: Checking the mathematics

To verify that the claims in the introduction are true, see the below python code.

First, we set up our symbolic variables and define $c$, $s$ and $\theta$ in terms of them. We then use `sympy` to find the first and second derivatives of $\theta$ with respect to time. Finally we construct the ODE using $\theta$ and it's derivatives.

We then run the following checks:

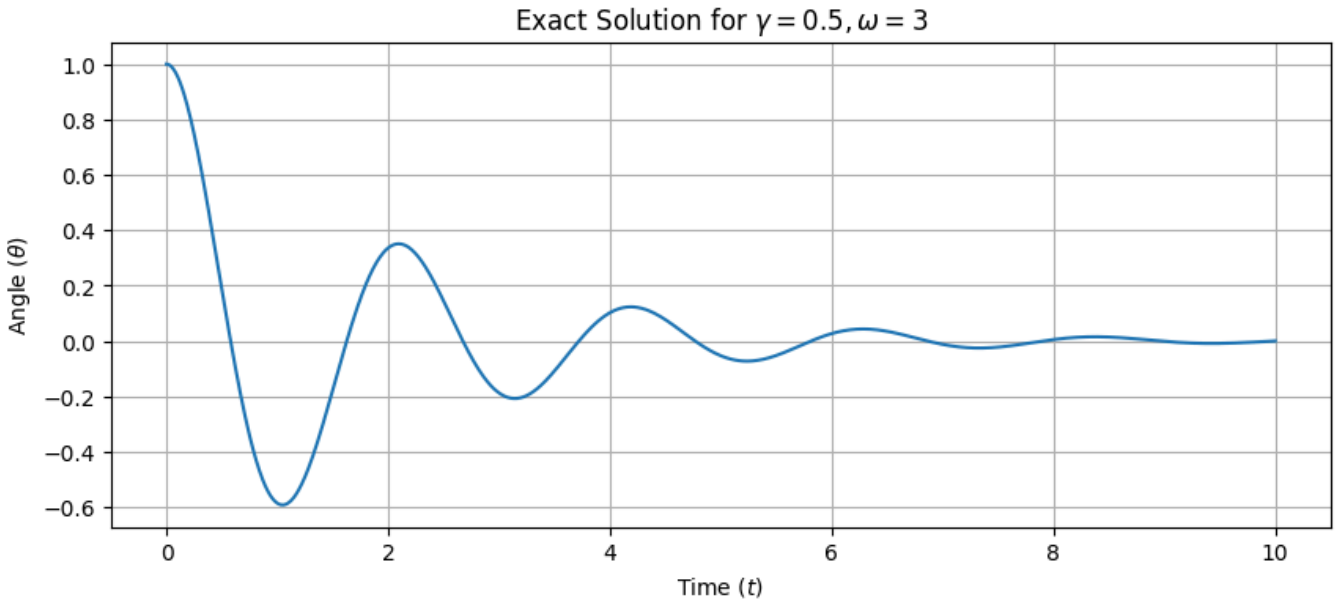
1. At time $t = 0$ we expect our solution to take the value $\theta_0$. To do this we use `.subs` to substitute the value $0$ for $t$, and then `.simplify` to symbolically check we have $\theta_0$.
2. Similarly we expect $\dot{\theta}$ to produce $\eta_0$ when we substitute $t = 0$.
3. We check that $\theta$ and it's derivatives subbed into the ODE simplifies to $0$, in other words, satisfy the differential equation.

In [2]:
```
# Setup some symbols
ts, gamma, omega_d = sp.symbols('t gamma omega_d', real=True)

# Initial condition symbols
theta_0, eta_0 = sp.symbols('theta_0 eta_0', real=True)

# Define c(t) and s(t)
c = sp.exp(-gamma*ts) * sp.cos(omega_d*ts)
s = sp.exp(-gamma*ts) * sp.sin(omega_d*ts)

# Define theta(t), this is the solution to the ODE
theta = theta_0 * c + ((eta_0 + gamma*theta_0)/omega_d) * s

# Derivatives
theta_dot = sp.diff(theta, ts)
theta_ddot = sp.diff(theta_dot, ts)

# ODE: theta'' + 2\gamma theta' + (\omega_d^2 + \gamma^2) theta
ode_expr = theta_ddot + 2*gamma*theta_dot + (omega_d**2 + gamma**2)*theta


# ---- Checks ----
# \theta_0 should be what we get at t=0
print("Check 1: theta(0) = theta_0")
print(sp.simplify(theta.subs(ts, 0)))

# We expect \eta_0 as the initial angular velocity
print("\nCheck 2: theta'(0) = eta_0")
print(sp.simplify(theta_dot.subs(ts, 0)))

# ODE with theta and it's derivatives should simplify to 0,
# which means theta solves the ODE.
print("\nCheck 3: ODE satisfied (should be 0)")
print(sp.simplify(ode_expr))
```

```
Check 1: theta(0) = theta_0
theta_0

Check 2: theta'(0) = eta_0
eta_0

Check 3: ODE satisfied (should be 0)
0
```

### 2.2: Plotting the solution

In [3]:
```python
def cos_exp(ts, gamma, omega_d):
    return np.exp(-gamma*ts) * np.cos(omega_d*ts)

def sin_exp(ts, gamma, omega_d):
    return np.exp(-gamma*ts) * np.sin(omega_d*ts)


def plot_exact(gamma, omega_d, theta0, eta0, t_max=10, num_points=1000):
    """
    Plot the exact solution to the discussed ODE for given parameters.

    :param gamma: Damping coefficient
    :param omega_d: Damped frequency
    :param theta0: Initial angle
    :param eta0: Initial angular velocity
    :param t_max: Maximum time to plot
    :param num_points: Number of points in the plot
    """
    ts = np.linspace(0, t_max, num_points)
    cs = cos_exp(ts, gamma, omega_d)
    ss = sin_exp(ts, gamma, omega_d)
    thetas = theta0 * cs + ((eta0 + gamma*theta0)/omega_d) * ss

    plt.figure(figsize=(10, 4))
    plt.plot(ts, thetas)
    plt.title(f'Exact Solution for $\\gamma={gamma}, \\omega_d={omega_d}$')
    plt.xlabel('Time ($t$)')
    plt.ylabel('Angle ($\\theta$)')
    plt.grid()
    plt.show()


# plot_exact(gamma=0.5, omega_d=3, theta0=1, eta0=0)
```




Above is the plot of $\theta$ against time with parameters $\gamma = 0.5$, $\omega = 3$, $\theta_0 = 1$ and $\eta_0 = 0$. It shows a clear damped oscillating pattern. "oscillating" here describes the solution moving up and down in a regular fashion, like a wave, and the "damped" part describes the magnitude of this wave gradually decreasing.

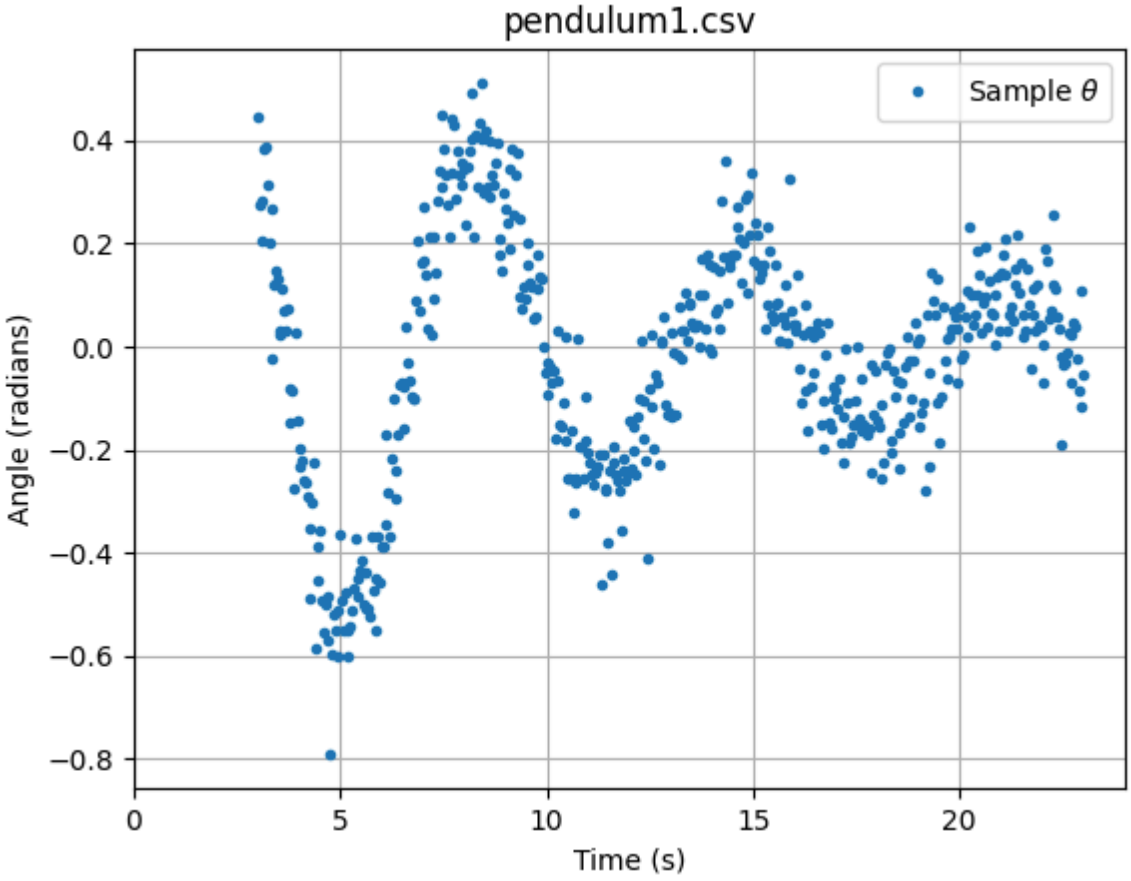
# Part 3: Fitting a linear model

### 3.1: Plotting the data

In [4]:

```
# Read the CSV file into a DataFrame
df = pd.read_csv("pendulum1.csv")

# Inspect the head of the dataframe
print("Head of 'pendulum1.csv':")
print(df.head())

# Print column names and data types
print("\nColumn names:")
print(*list(df.columns), sep=", ")
print("\nData types:")
print(df.dtypes)

# ---- Plotting ----
plt.figure()

# Create plot from columns 1 (time) and 2 (angle) of the dataframe
plt.plot(df.iloc[:, 1], df.iloc[:, 2], ".", label="Sample $\\theta$")

# Force x-axis to start at 0
plt.xlim(left=0)

# Add labels and legend
plt.xlabel("Time (s)")
plt.ylabel("Angle (radians)")
plt.title("pendulum1.csv")
plt.legend()
plt.grid()

plt.show()
```

```
Head of 'pendulum1.csv':
   Unnamed: 0     t     theta
0           0  3.00  0.445173
1           1  3.04  0.275452
2           2  3.08  0.206092
3           3  3.12  0.284490
4           4  3.16  0.382913

Column names:
Unnamed: 0, t, theta

Data types:
Unnamed: 0      int64
t             float64
theta         float64
dtype: object
```




Plot of data from `pendulum1.csv` file. We see there is an oscillatory relationship between $\theta$ (angle of the pendulum in radians) and time $t$. There is also a significant amount of *noise* in this data.

**3.2: Fitting a model**

We will now fit a model to this data by using a two parameter linear model described as follows

$$F(t;\beta_0,\beta_1) \equiv \beta_0\phi_0(t) + \beta_1\phi_1(t),$$

where $\beta_0$ and $\beta_1$ are parameters, $\phi_0(t) = c(t)$ and $\phi_1(t) = s(t)$ are the functions from part 1.

By inspecting the $\theta(t)$ expression we can determine the values of $\theta_0$ and $\eta_0$ based off $\beta_0$ and $\beta_1$.

$$\theta(t) = \theta_0 c(t) + \left(\frac{\eta_0 + \gamma\theta_0}{\omega_d}\right) s(t)$$

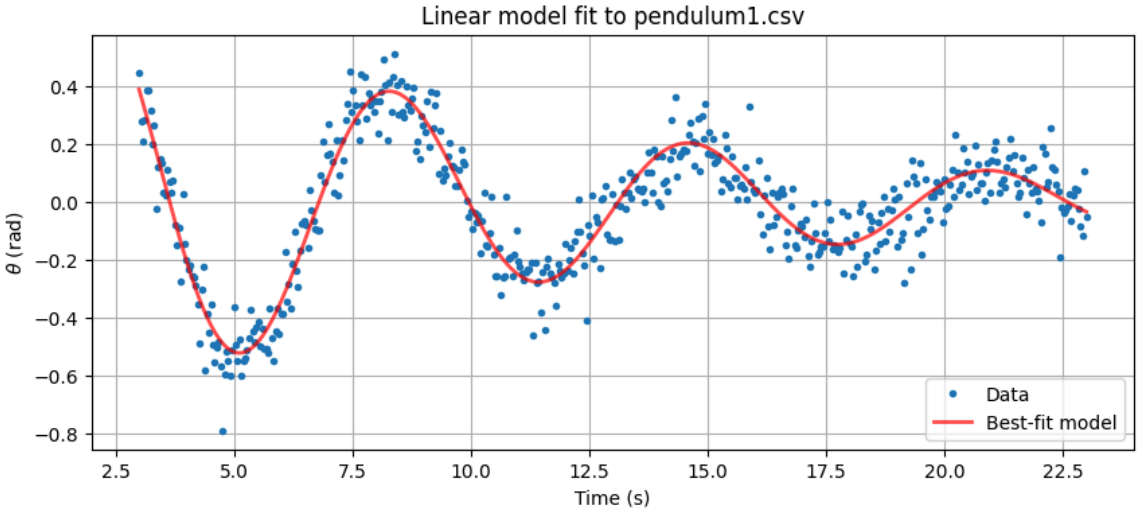
Note that $\theta_0$ is the coefficient of cosine in this expression, and that $\beta_0$ is the coefficient of cosine in our linear model, so we deduce that $\theta_0 = \beta_0$. Similarly, the coefficient of sine in the expression delivers us our expression for $\eta_0$.

$$\beta_1 = \frac{\eta_0 + \gamma\theta_0}{\omega_d} \implies \eta_0 = \beta_1\omega_d - \gamma\theta_0$$

In [5]:

```
# This data is generated from a pendulum with known natural frequency 1
# and damping coefficient 0.1.
gamma = 0.1
omega = 1
omega_d = np.sqrt(omega**2 - gamma**2)


# Extract data from the dataframe
df = pd.read_csv("pendulum1.csv")
ts = df.iloc[:, 1].values      # t
thetas = df.iloc[:, 2].values  # theta


# 1. Build X (size N x 2) with phi_0 = c(t) and phi_1 = s(t)
X = np.column_stack([cos_exp(ts, gamma, omega_d), sin_exp(ts, gamma, omega_
d)])

# 2. Build A = X^T X and b = X^T theta
A = X.T @ X
b = X.T @ thetas


# 3. Solve for beta = [beta0, beta1]
beta = np.linalg.solve(A, b)
beta0, beta1 = beta
print(f"beta0 = {beta0}, beta1 = {beta1}")


# 4. Recover theta0 and eta0
theta0_hat = beta0
eta0_hat = beta1 * omega_d - gamma * theta0_hat
print(f"Estimated theta0 = {theta0_hat:.6g}")
print(f"Estimated eta0   = {eta0_hat:.6g}")


# ------ Plot  ------
prediction_data = X @ beta

plt.figure(figsize=(10,4))
plt.plot(ts, thetas, '.', label='Data')
plt.plot(ts, prediction_data, '-', label='Best-fit model', linewidth=2, col
or='red', alpha=0.7)
plt.xlabel('Time (s)')
plt.ylabel('$\\theta$ (rad)')
plt.title(f'Linear model fit to pendulum1.csv')
plt.legend(loc='lower right')
plt.grid(True)

plt.show()
```

```
beta0 = -0.40662775771420345, beta1 = 0.7767633670216031
Estimated theta0 = -0.406628
Estimated eta0   = 0.813533
```

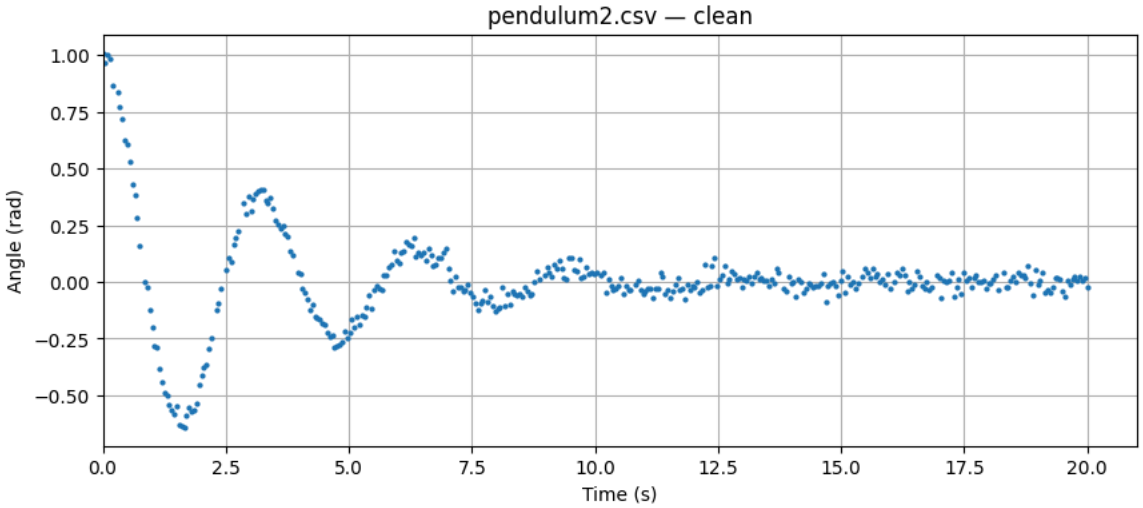



As the natural frequency $\omega$ and damping coefficient $\gamma$ are known for this pendulum, we can create a well-fitting model.

## 4. Parameter estimation by curve fitting

**Task 4.1: Cleaning the data**

In [6]:
```
# Load data
df2 = pd.read_csv("pendulum2.csv")
t_col = df2.iloc[:, 0]
theta_col = df2.iloc[:, 1]

# Remove rows with missing values
df2 = df2.dropna()


# Remove implausible values of theta
mask = np.abs(theta_col) <= (np.pi / 2)
df2_clean = df2.loc[mask]


# Convert cleaned data to numpy array
ts2 = df2_clean.iloc[:, 0].values
thetas2 = df2_clean.iloc[:, 1].values


# ------ Plot  ------
plt.figure(figsize=(10,4))
plt.plot(ts2, thetas2, '.', markersize=4)
plt.xlabel("Time (s)")
plt.ylabel("Angle (rad)")
plt.title("pendulum2.csv — clean")
plt.grid(True)
plt.xlim(left=0)
plt.show()
```




The cleaned data from `pendulum2.csv` is consistent with a underdamped pendulum. We can clearly see an oscillatory motion which has a gradually decreasing amplitude. The cleaning has removed erroneous or extreme values which may interfere with the next parts.

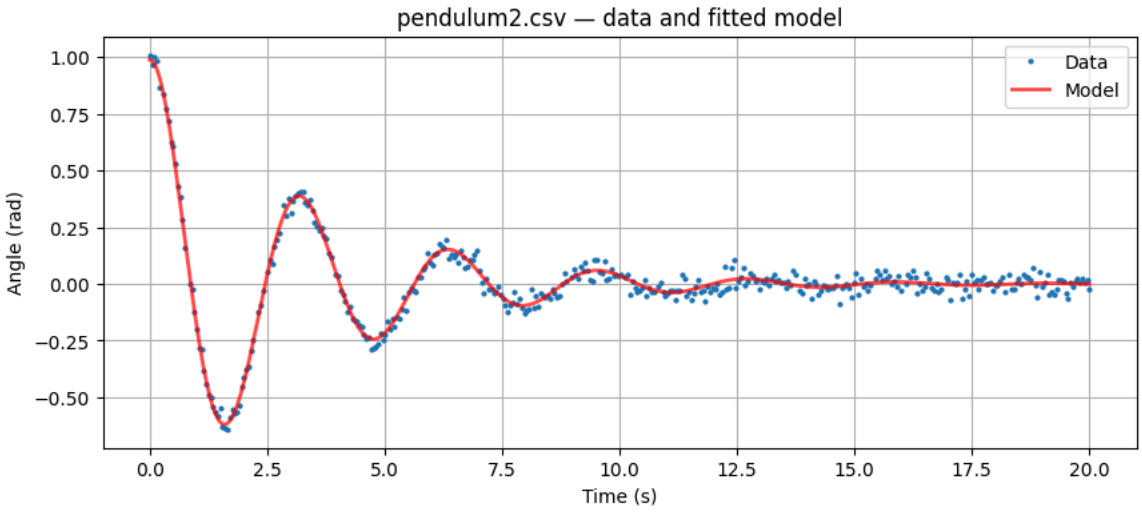
**Task 4.2: Estimating parameters**

Explain the method here.

In [7]:

```
def exact_model(t, gamma, omega_d,  theta0, eta0):
    """Right-hand side of equation (C): theta(t) for parameters gamma, omeg
a_d, theta0, eta0."""
    t = np.asarray(t)
    c = np.exp(-gamma * t) * np.cos(omega_d * t)
    s = np.exp(-gamma * t) * np.sin(omega_d * t)
    return theta0 * c + ((eta0 + gamma * theta0) / omega_d) * s


# ------ Initial guess for parameters ------
gamma0 = 0.05                    # A small positive quantity
theta0_0 = thetas2[0]            # Approximately the first angle measurement
eta0_0 = 0.0                     # Around 0
T_est = 3                        # Read from graph
omega_d0 = 2 * np.pi / T_est


# --------- Compute parameter estimates using curve_fit ---------
p0 = [gamma0, omega_d0, theta0_0, eta0_0]
params_est, param_cov = sc.optimize.curve_fit(exact_model, ts2, thetas2, p0
=p0,  maxfev=10000)

gamma_hat, omega_d_hat, theta0_hat, eta0_hat = params_est

print("Estimated parameters:")
print(f" gamma = {gamma_hat:.6}")
print(f" omega_d = {omega_d_hat:.6}")
print(f" theta0 = {theta0_hat:.6}")
print(f" eta0 = {eta0_hat:.6}")


# ------ Plot  ------
t_fit = np.linspace(ts2.min(), ts2.max(), 2000)
# Get y values from exact model, using our estimated parameters
y_fit = exact_model(t_fit, gamma_hat, omega_d_hat, theta0_hat, eta0_hat)

plt.figure(figsize=(10,4))
plt.plot(ts2, thetas2, '.', markersize=4, label='Data')
plt.plot(t_fit, y_fit, '-', linewidth=2, label='Model', color='red', alpha=
0.7)
plt.xlabel('Time (s)')
plt.ylabel('Angle (rad)')
plt.title('pendulum2.csv — data and fitted model')
plt.legend()
plt.grid(True)
plt.show()
```

```
Estimated parameters:
 gamma = 0.294853
 omega_d = 1.98124
 theta0 = 0.988428
 eta0 = 0.0217437
```




We have achieved a high quality of fit using this method. As we are now working directly with the solution to the ODE, and have hte ability to modify all four parameters used in the solution, `scipy.optimize` is able to pick estimates which cause the model to fit the data exceptionally well. Note that we are able to do this because the initial conditions for the parameters can be estimated directly from the data, or by heuristic knowledge of the pendulum setup (such as knowing $\eta_0 \approx 0$).

***PARAMETER ESTIMATES***

$$\begin{aligned}\hat{\gamma} &= 0.294853\\ \hat{\omega}_d &= 1.98124\\ \hat{\theta}_0 &= 0.988428\\ \hat{\eta}_0 &= 0.0217437\end{aligned}$$

**Task 4.3: Applying the estimates**

We can now use our parameter estimates to guess the length of the pendulum used to record the data in `pendulum2.csv` . As discussed in part 1, the pendulum length, $\ell$, is modelled in the relation $\ell = g/\omega^2$, where $\omega = \sqrt{\omega_d^2 + \gamma^2}$.

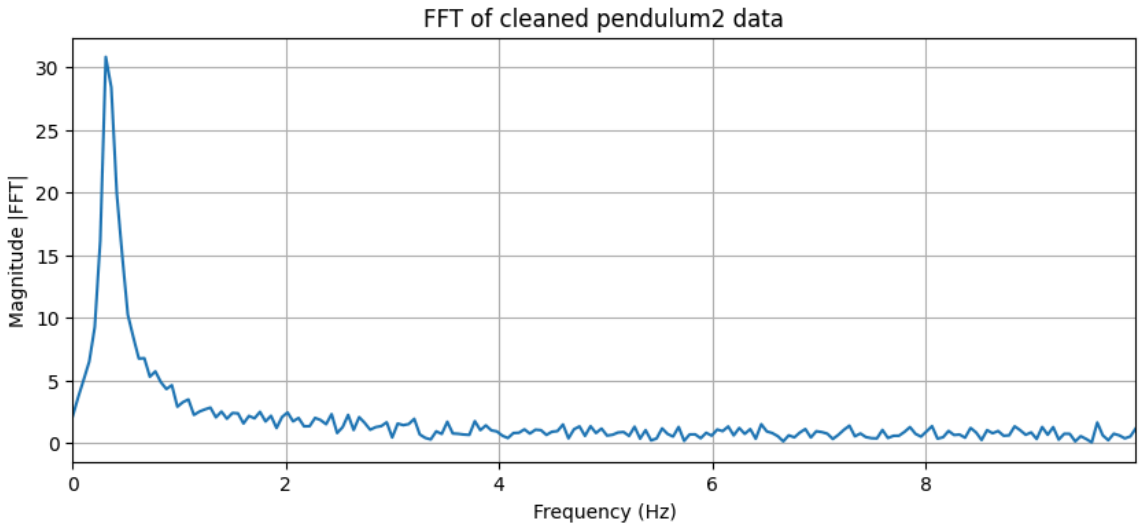
$$\begin{aligned}\omega &= \sqrt{\omega_d^2 + \gamma^2} = 2.054304\\ \ell = g/\omega^2 &= \boxed{2.324554\ \ (\mathrm{m})}\end{aligned}$$

This is quite a large pendulum, but perfectly reasonable physically!

# 5. Signal Processing

### Task 5.1: Fourier transform

In [8]:
```
# Choose time step from data
dt = np.median(np.diff(ts2))


# Compute FFT and frequency bins
Y = np.fft.rfft(thetas2)  #
freqs = np.fft.rfftfreq(thetas2.size, d=dt)   # Frequencies in Hz
mag = np.abs(Y)  # FFT gives complex numbers; we take the magnitude

# --------- Plot ---------
plt.figure(figsize=(10,4))
plt.plot(freqs, mag, '-')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude |FFT|')
plt.title('FFT of cleaned pendulum2 data')
plt.xlim(0, freqs.max())
plt.grid(True)
plt.show()


# Find dominant frequency
max_index = np.argmax(mag)
f_star = freqs[max_index]  # Max frequency in Hz
omega_d_star = 2 * np.pi * f_star  # Max angular frequency in rad/s


print(f"Dominant frequency f* = {f_star:.6} Hz")
print(f"Angular frequency omega_d* = {omega_d_star:.6} rad/s")

# Compare with curve_fit result
print(f"Task 4.2 omega_d = {omega_d_hat:.6} rad/s")
rel_diff = 100 *(omega_d_star - omega_d_hat) / omega_d_hat
print(f"Percent difference = {rel_diff:.4}%")
```




```
Dominant frequency f* = 0.310104 Hz
Angular frequency omega_d* = 1.94844 rad/s
Task 4.2 omega_d = 1.98124 rad/s
Percent difference = -1.656%
```

The above simulation uses a discrete Fourier transform (using the "Fast Fourier Transform" algorithm included in `NumPy`) to analyse the constituent frequencies encoded in the data. Put briefly, this algorithm converts the data into the *frequency domain* (as opposed to the time domain), meaning we see frequency on the $x$ axis, and the "strength" of the frequency on the $y$.

By noting where there are peaks in the graph, we identify the dominant frequencies of the underlying data. Note that we expect there to be a bit of "noise" in our graph as this representation is derived from noisy data its self, hence the bulk around the peak and the spikiness.

It's quite clear from this graph that the data from `pendulum2.csv` has a single dominant frequency, which corresponds to the frequency at which the pendulum is oscillating. We can convert this measured dominant frequency $f^*$ to measured angular frequency $\omega_d^*$ and compare it to the angular frequency we estimated in part 4.2:

$$\begin{aligned} f^* &= 0.310104 \ \ (\text{Hz}) \\ \omega_d^* &= 1.94844 \ \ (\text{rad/s}) \\ \omega_d &= 1.98124 \end{aligned}$$

$\omega_d^*$ is within $1.656\%$ of $\omega_d$.

## 6. Further exploration

#### Task 6.1: Stroboscopic plots

We now consider a system where the angular velocity is forced to be at some value. In our case we fix the pendulum at some expression with respect to time.

$$\ddot{\theta} + 2\gamma\dot{\theta} + \omega^2 \sin(\theta) = \alpha \cos(\Omega t)$$

This has the effect of shaking the pendulum by an external force with frequency $\Omega$, or a perpendicular force applied to the bob of the pendulum.

We then rewrite this second order ODE by defining $\eta = \dot{\theta}$ and splitting it into two coupled first-order ODEs:

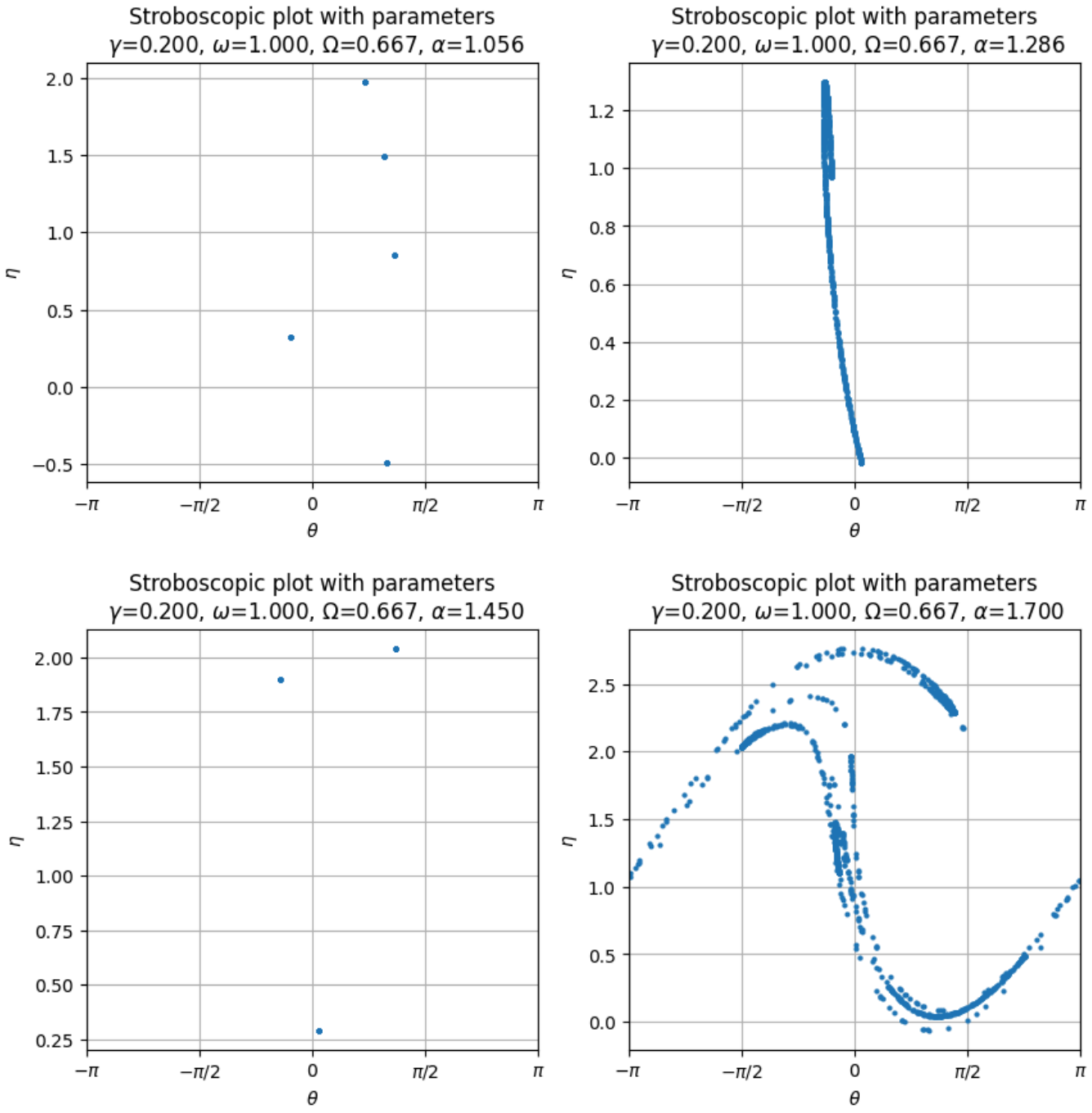
$$\begin{aligned} \dot{\theta} &= \eta \\ \dot{\eta} + 2\gamma\eta + \omega^2 \sin(\theta) &= \alpha \cos(\Omega t) \end{aligned}$$

We can then apply `sc.integrate.solve_ivp()` to find a numerical solution to $(\theta, \eta)$ given the parameters $\gamma$ (damping coefficient), $\omega$ (natural frequency of the pendulum), $\Omega$ (period of driving force) and $\alpha$ (magnitude of driving force). All simulations start with initial conditions $(\theta, \eta) = (0, 0)$.

In order to analyse the chaos in our system, we will create a "stroboscopic" plot. This means that we are recording the state of pendulum at regularly placed intervals, much like how a strobe light illuminates a scene periodically. By taking a snapshot of the state at an interval that is proportional to the period of the driving force ($\Omega$), we should be able to notice when the pendulum is swinging in a regular, periodic manor. This will manifest in our graph as seeing only a limited number of points, as the readings are identical and the points are intersecting. These points are called the *steady states* of the system. If we see a more interesting pattern, that means that the readings from the pendulum were slightly different every time, so the system is exhibiting a chaotic motion.

In [9]:
```
def stroboscopic_plot(gamma=1/5, omega=1, Omega=2/3, alpha=1.8, t_max=4000,
ax=None):
    """
    Generate a stroboscopic plot for the given parameters. The plot samples
    the state of the pendulum at multiples of the driving period T = 2*pi/O
mega,
    and discards the transient behaviour by only plotting from t=500 to t=t
_max.

    :param gamma: Damping coefficient
    :param omega: Natural frequency of the pendulum
    :param Omega: Driving frequency
    :param alpha: Driving amplitude
    :param t_max: Maximum time to simulate
    :param ax: Optional matplotlib axis to plot on. If None, a new figure a
nd
                axis will be created.
    """

    if ax is None:
        plot, ax = plt.subplots(figsize=(8, 6))

    t0 = 0
    t1 = 500  # Threshold time to discard transient behaviour
    t2 = t_max

    # First-order system
    def system(t, y):
        theta, eta = y

        # System described above
        dtheta_dt = eta
        deta_dt = -2 * gamma * eta - omega**2 * np.sin(theta) + alpha * np.
cos(Omega * t)  # (=0)
        return [dtheta_dt, deta_dt]


    # Record at multiples of T = 2*pi/Omega
    T = 2 * np.pi / Omega

    # We only want multiples of T from t1 to t2,
    n_start = int(np.ceil(t1 / T))
    n_end = int(np.floor(t2 / T))
    t_eval = np.arange(n_start, n_end + 1) * T


    # Solve from t0 to t2. Using [0, 0] as initial condition
    sol = sc.integrate.solve_ivp(
        system,
        (t0, t2),
        [0.0, 0.0],
        t_eval=t_eval,
        method='RK45',
        rtol=1e-8,
        atol=1e-8,
    )

    theta = sol.y[0]
    eta = sol.y[1]
```

```
    # Fold theta into the range (-pi, pi]
    theta_folded = (theta + np.pi) % (2 * np.pi) - np.pi


    ax.plot(theta_folded, eta, '.', markersize=4)
    ax.set_title(f'Stroboscopic plot with parameters\n $\\gamma$={gamma:.3
f}, $\\omega$={omega:.3f}, $\\Omega$={Omega:.3f}, $\\alpha$={alpha:.3f}')
    ax.set_xlabel('$\\theta$')
    ax.set_ylabel('$\\eta$')

    # Horizontal axis markings at 0, +/- pi/2 and +/- pi,
    pi_ticks = [-np.pi, -np.pi/2, 0, np.pi/2, np.pi]
    pi_labels = ['$-\\pi$', '$-\\pi/2$', '0', '$\\pi/2$', '$\\pi$']
    ax.set_xticks(pi_ticks)
    ax.set_xticklabels(pi_labels)
    ax.set_xlim(-np.pi, np.pi)
    ax.grid(True)


# Expect ~ 20s total
"""
fig, axes = plt.subplots(2, 2, figsize=(10, 10))
fig.subplots_adjust( hspace=0.35 )  # Title was clipping

stroboscopic_plot(alpha=1.056, t_max=4000, ax=axes[0, 0])
stroboscopic_plot(alpha=1.286, t_max=10000, ax=axes[0, 1])
stroboscopic_plot(alpha=1.45, t_max=4000, ax=axes[1, 0])
stroboscopic_plot(alpha=1.7, t_max=10000, ax=axes[1, 1])
""";
```




We have selected some values of $\alpha$ and created stroboscopic plots for the system. We clearly see that there are a number of different regimes that the behaviour of the pendulum can fall in depending on this parameter.

The two plots on the left indicate a relatively reciprocal pattern - the pendulum must be swinging in a way that repeats itself so that the readings we take from it overlap perfectly. We see plot (0, 0) has $5$ distinct readings of $\theta$ over it's duration and plot (0, 1) has three.

The rightmost plots are more interesting. We see the top one has some sort of pattern in $(\theta, \eta)$ space, but this corresponds to something less pretty in terms of the pendulum. We are getting a slightly different reading from the pendulum at every time interval, and although those readings are periodic it means the actual state of the pendulum is slightly different every time. It would be most interesting to visualise what the pendulum is doing here. We would expect that due to the periodic nature of the $\theta$, $\eta$ parameters, there is still some periodicity in the swing of the pendulum.

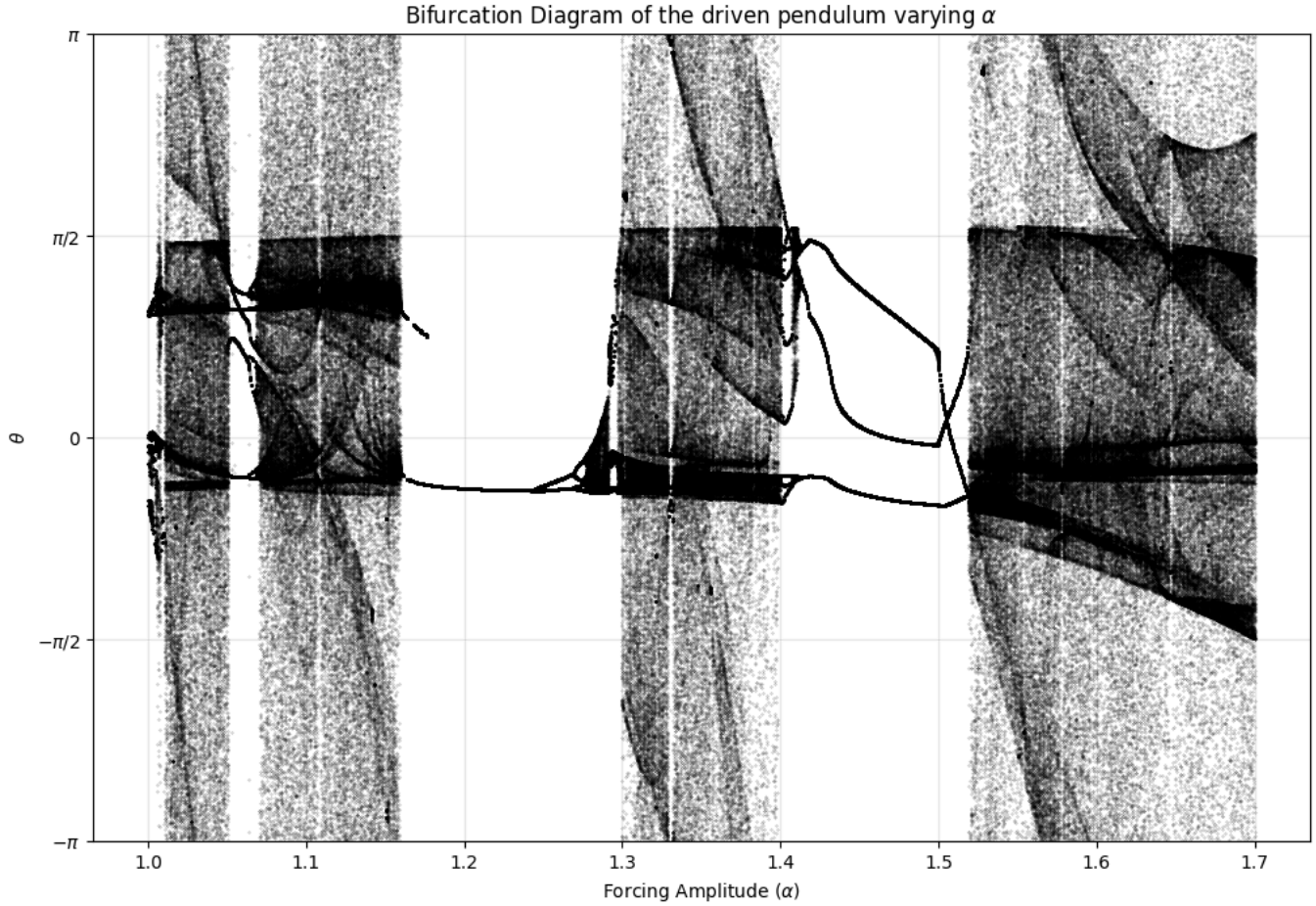
The bottom right plot displays the pendulum in a chaotic state. Whilst there is some structure to the plot, the majority of readings are distinct to prior readings, meaning that the pendulum is reliably in a different state every time the measurement is taken. We should expect the pendulum to appear "chaotic" and unpredictable as it swings.

**Task 6.2: Investigating pendulum states**

In this section we further analyse the effect of changing the $\alpha$ parameter on the steady state behaviour of the pendulum. We approach this by creating a plot that varies $\alpha$ on the $x$-axis, and records the values of $\theta$ attained after transient behaviour has settled. The result will be a plot where we can easily see regions of $\alpha$ that lead to a nice steady state (like in plot (0, 0) and (0, 1) above), and those values which lead to chaos.

In [10]:
```
def bifurcation_diagram(gamma=1/5, omega=1, Omega=2/3, alpha_min=1.0, alpha
_max=1.5, num_alphas=150, t_max=1500):
    """
    Create a bifurcation diagram with respect to alpha by recording theta v
alues
    attained in the steady state of the system. Alpha is plotted on the x-a
xis,
    and on the y, theta values attained at multiples of the driving period.

    :param gamma: Damping coefficient
    :param omega: Natural frequency of the pendulum
    :param Omega: Driving frequency
    :param alpha_min: Minimum value of alpha to simulate
    :param alpha_max: Maximum value of alpha to simulate
    :param num_alphas: Number of alpha values to simulate between alpha_min
and alpha_max
    :param t_max: Maximum time to simulate for each alpha.
    """

    # Define an array of alpha values to try
    alphas = np.linspace(alpha_min, alpha_max, num_alphas)

    t0 = 0
    t1 = 2000  # Discard the first 500 seconds as transient
    t2 = t_max

    # Record at multiples of T = 2*pi/Omega
    T = 2 * np.pi / Omega
    n_start = int(np.ceil(t1 / T))
    n_end = int(np.floor(t2 / T))
    t_eval = np.arange(n_start, n_end + 1) * T  # Only multiples of T

    plt.figure(figsize=(12, 8))

    A = []
    T = []

    # Iterate over the different forcing amplitudes
    for alpha in alphas:
        # Define the system for the current alpha
        def system(t, y):
            theta, eta = y
            dtheta_dt = eta
            deta_dt = -2 * gamma * eta - omega**2 * np.sin(theta) + alpha *
np.cos(Omega * t)
            return [dtheta_dt, deta_dt]

        # Solve the ODE
        sol = sc.integrate.solve_ivp(
            system,
            (t0, t2),
            [0.0, 0.0],  # Starting from rest
            t_eval=t_eval,
            method='RK45',
            rtol=1e-6,
            atol=1e-6
        )

        theta = sol.y[0]
```

```
        # Fold theta into the range (-pi, pi]
        theta_folded = (theta + np.pi) % (2 * np.pi) - np.pi
        all_alpha = [alpha] * len(theta_folded)

        A.append(all_alpha)
        T.append(theta_folded)

        # Scatter plot for this specific alpha
        plt.plot(all_alpha, theta_folded, '.k', markersize=0.5, alpha=0.5)

    np.savez('bifurcation_data.npz', T=T, A=A)

    # ------ Plot -------
    plt.title(f'Bifurcation diagram of the driven pendulum varying $\\alpha
$\n' \
    f'Parameters: $\\gamma$={gamma:.3f}, $\\omega$={omega:.3f}, $\\Omega$=
{Omega:.3f}')
    plt.xlabel('Forcing Amplitude ($\\alpha$)')
    plt.ylabel('$\\theta$')

    pi_ticks = [-np.pi, -np.pi/2, 0, np.pi/2, np.pi]
    pi_labels = ['$-\\pi$', '$-\\pi/2$', '0', '$\\pi/2$', '$\\pi$']
    plt.yticks(pi_ticks, pi_labels)
    plt.ylim(-np.pi, np.pi)
    plt.grid(True, alpha=0.3)
    plt.show()


# Expect ~30 mins
# bifurcation_diagram(alpha_min=1, alpha_max=1.7, num_alphas=550 , t_max=30
00)
```




Above is a high resolution render of the bifurcation plot of the variable $\alpha$ between the values $1$ and $1.7$. We can clearly see now how the system changes between regimes of periodic steady-states and chaotic states.

The areas where the graph appears stringy and continuous are values of $\alpha$ which lead to oscillations of the pendulum that appear periodic when viewed stroboscopically. We can also see exactly how many steady states there are by viewing how many $\theta$ values the graph takes at a given $\alpha$. For instance, $1.4 \leq \alpha \leq 1.5$ is a regime where $3$ values of $\alpha$ are constantly revisited, and at around $\alpha = 1.2$ we have just one.

There is another feature here that is quite eye-catching. At around $\alpha = 1.18$ there is an abrupt stop to one of the steady states of $\theta$. This means that once alpha passes a certain quantity, there is a considerable change to how the dynamics function, causing this steady state to completely disappear. I believe, after consulting my notes from MPS236, that what we are seeing here is a *saddle-node* bifurcation, but we just can't see the unstable steady state. A saddle node bifurcation is characterised by a stable node and an unstable node coming together and annihilating, leaving no stability in it's wake. Here, we see that this stable point has disappeared in a similar way.

Lets visualise the pendulums at the alpha values described in section 6.1 to try and convince ourselves of what is happening, and to see what these different modes of pendulum motion actually look like.

In [11]:

```
plt.rcParams['animation.embed_limit'] = 100
plt.rcParams["animation.html"] = "jshtml"


def animate_pendulum(gamma=1/5, omega=1, Omega=2/3, alpha=1.45,
                     theta0=0.0, eta0=0.0, t_duration=30, fps=30,
                     playback_speed=2.0, force_arrow_scale=0.3, transient_t
ime=500.0,
                     show_strobe_dots=True, skip_transient=True):
    """
    Simulates and animates a forced pendulum with a driving force vector.
    Returns an HTML object that can be displayed interactively in a Jupyter
Notebook.

    :param gamma: Damping coefficient
    :param omega: Natural frequency of the pendulum
    :param Omega: Driving frequency
    :param alpha: Forcing amplitude. If float, remains constant and no tran
sients are skipped.
                If tuple (alpha1, alpha2),  shows alpha1 for 1/10 of t_duratio
n, then snaps to alpha2.
    :param theta0, eta0: Initial conditions
    :param t_duration: Duration of the visible animation in seconds
    :param fps: Visual frames rendered per second of real viewing time
    :param playback_speed: Multiplier for simulation speed
    :param force_arrow_scale: Visual length multiplier for the driving forc
e arrow
    :param transient_time: Seconds to simulate silently before the animatio
n starts (only if alpha is a tuple).
    :param show_strobe_dots: Whether to display stroboscopic dots
    :param skip_transient: Skips first transient_time seconds of simulation
if True.
    """
    # Stores if we are to change alpha part way though animation
    is_sweep = isinstance(alpha, (list, tuple))

    # Calculate some time values
    t0 = transient_time if skip_transient else 0.0
    t_end = t0 + t_duration
    T = 2 * np.pi / Omega
    flash_threshold = 0.1 * playback_speed


    # Helper to calculate the current alpha value based on simulation time
    def get_alpha(t):
        if is_sweep:
            t_switch = t0 + (t_duration / 10.0)
            return alpha[0] if t < t_switch else alpha[1]
        return alpha

    # Define the system of ODEs
    def system(t, y):
        theta, eta = y
        dtheta_dt = eta
        deta_dt = -2 * gamma * eta - omega**2 * np.sin(theta) + get_alpha
(t) * np.cos(Omega * t)
        return [dtheta_dt, deta_dt]
```

```
    # --- Solve 1: For the continuous animation frames ---
    total_frames = int((t_duration * fps) / playback_speed)
    t_eval_frames = np.linspace(t0, t_end, total_frames)

    sol_frames = sc_int.solve_ivp(
        system,
        (0, t_end),
        [theta0, eta0],
        t_eval=t_eval_frames,
        method='RK45',
        rtol=1e-8,  # Bumped up to 1e-8 to perfectly match your stroboscopi
c function!
        atol=1e-8
    )
    theta_frames = sol_frames.y[0]
    x_frames = np.sin(theta_frames)
    y_frames = -np.cos(theta_frames)

    # --- Solve 2: For the exact stroboscopic dots ---
    # This is required for precision.
    strobe_times = np.array([])
    strobe_xs = np.array([])
    strobe_ys = np.array([])

    if show_strobe_dots:
        n_start = int(np.ceil(t0 / T))
        n_end = int(np.floor(t_end / T))
        if n_end >= n_start:
            # Tell the solver to evaluate exactly at the multiples of T
            t_eval_strobe = np.arange(n_start, n_end + 1) * T
            sol_strobe = sc_int.solve_ivp(
                system,
                (0, t_end),
                [theta0, eta0],
                t_eval=t_eval_strobe,
                method='RK45',
                rtol=1e-8,
                atol=1e-8
            )
            theta_strobe = sol_strobe.y[0]
            strobe_xs = np.sin(theta_strobe)
            strobe_ys = -np.cos(theta_strobe)
            strobe_times = t_eval_strobe


    # --- Setup Plot ---
    fig, ax = plt.subplots(figsize=(5, 5))
    plt.tight_layout(rect=[0, -0.31, 1, 1.23], h_pad=0.2)

    max_alpha = max(alpha) if is_sweep else alpha
    ax.set_xlim(-1.5, 1.5)
    ax.set_ylim(-1.5, 1.5)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.3)

    title_str = f"with parameter change: {alpha[0]} -> {alpha[1]}" if is_sw
eep else ""
    param_str = f"$\\gamma={gamma:.2f}, \\omega={omega:.2f}, \\Omega={Omeg
a:.2f}$"
```

```
    ax.set_title(f'Forced pendulum {title_str}\n{param_str}')
    ax.plot([0], [0], 'ko', markersize=4)

    # Plot features
    strobe_dots, = ax.plot([], [], 'ro', markersize=8, alpha=0.6, zorder=2)
    rod, = ax.plot([], [], '-', lw=3, color='royalblue', zorder=3)
    bob, = ax.plot([], [], 'o', markersize=15, color='darkorange', zorder=
4)
    info_text = ax.text(0.05, 0.92, '', transform=ax.transAxes, fontsize=1
2, fontfamily='monospace')
    force_arrow = ax.quiver(0, 0, 0, 0, color='crimson', scale=1, scale_uni
ts='xy',
                            angles='xy', width=0.012, pivot='tail', zorder=
5)

    # Initialise animation features.
    def init():
        rod.set_data([], [])
        bob.set_data([], [])
        info_text.set_text('')
        force_arrow.set_offsets(np.array([[0, 0]]))
        force_arrow.set_UVC(0, 0)
        strobe_dots.set_data([], [])
        return rod, bob, info_text, force_arrow, strobe_dots

    # Tells animator how to update to next frame.
    def update(frame):
        t = t_eval_frames[frame]
        th = theta_frames[frame]
        current_alpha = get_alpha(t)

        # Set rod and bob position
        rod.set_data([0, x_frames[frame]], [0, y_frames[frame]])
        bob.set_data([x_frames[frame]], [y_frames[frame]])

        # Flash text logic
        dist = abs(t - round(t / T) * T)
        if dist < flash_threshold:
            info_text.set_color('red')
            info_text.set_fontweight('bold')
        else:
            info_text.set_color('black')
            info_text.set_fontweight('normal')

        info_text.set_text(f'Time: {t:05.1f}s | α: {current_alpha:.4f}')

        # Arrow logic
        F_drive = current_alpha * np.cos(Omega * t)
        u = F_drive * np.cos(th) * force_arrow_scale
        v = F_drive * np.sin(th) * force_arrow_scale
        force_arrow.set_offsets(np.array([[x_frames[frame], y_frames[fram
e]]]))
        force_arrow.set_UVC(u, v)

        # Exact Stroboscopic dots logic
        if show_strobe_dots and len(strobe_times) > 0:
            valid_idx = strobe_times <= t
            strobe_dots.set_data(strobe_xs[valid_idx], strobe_ys[valid_id
```

```
x])

        return rod, bob, info_text, force_arrow, strobe_dots


    ani = FuncAnimation(fig, update, frames=len(t_eval_frames),
                        init_func=init, blit=True, interval=1000/fps)
    # ani.save(f'export.gif', writer='pillow', fps=24)
    plt.close(fig)

    return HTML(ani.to_jshtml())


# Expect ~3 min each
# display(animate_pendulum(alpha=1.056, t_duration=200, playback_speed=6.0,
skip_transient=True))
# display(animate_pendulum(alpha=1.286, t_duration=200, playback_speed=6.0,
skip_transient=True))
# display(animate_pendulum(alpha=1.45, t_duration=200, playback_speed=6.0,
skip_transient=True))
# display(animate_pendulum(alpha=1.7, t_duration=200, playback_speed=6.0, s
kip_transient=True))
```

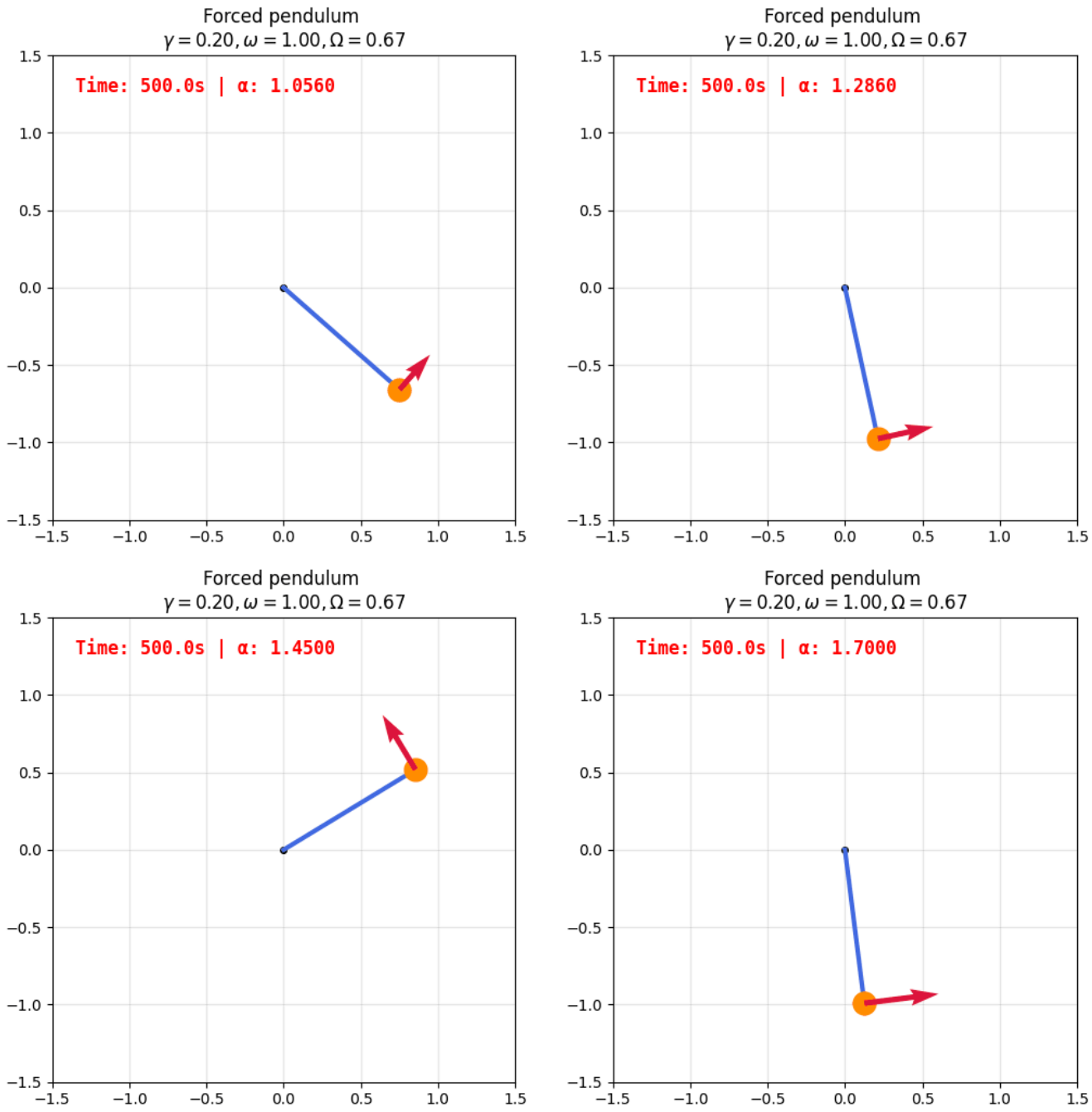
The above function simulates the differential equation we have defined and turns the output into an animation of a pendulum. We have added a few features to help depict what is happening.

The red arrow positioned at the bob of the pendulum is representing the driving force $\alpha\cos(\Omega t)$ that is being applied to the pendulum. The size and direction of the arrow represents the magnitude and direction of force being applied. Take a look at one of the plots and convince yourself that this arrow explains the movement of the pendulum.

These four plots correspond to the four stroboscopic plots above. In order to make it clear why exactly these plots look the way they do, we have added indications on the graph to the values of $t$ which are multiples of the sampling frequency, $T = 2\pi/\Omega$ - this is indicated by the text in the top left of the plot flashing red. Also added is a red dot at the time the pendulum state is read.

Looking at the stroboscopic plot with $\alpha = 1.056$ (top left), we can clearly see that there are $5$ distinct $\theta$ values that are attained. Similarly, the plot of the pendulum leaves a trail of precisely $5$ distinct red dots.

The $\alpha = 1.286$ plot reveals our pendulum is almost chaotic, but it's sampled angles are all bunched around the same range - which may be a distinct type of motion. The $\alpha = 1.7$ plot reveals a chaotic pendulum. The red dots appear with no pattern or certainty. As a further investigation, it would be interesting to see if this plot would theoretically span all values of $\theta$, or if it's attainable $\theta$ values are limited to a subset of $[0, 2\pi]$ - this would differentiate it from the top right plot.




Note that the rods of these pendulums do not change length when changing $\omega$.

## Acknowledgements


- https://sites.berry.edu/ttimberlake/wp-content/uploads/sites/37/2015/07/MaximaDDPworksheet.pdf (https://sites.berry.edu/ttimberlake/wp-content/uploads/sites/37/2015/07/MaximaDDPworksheet.pdf)
- https://en.wikipedia.org/wiki/Chaos_theory (https://en.wikipedia.org/wiki/Chaos_theory)
- MPS236 Semester 1 notes

**AI acknowledgement**

AI was used in the creation of this document, including it's use for generating code.

- Gemini 3.1 Pro (High) was used to help generate code across the document. Easier tasks were deferred to Gemini Flash to conserve usage. In particular, AI was heavily used in part 6 of the project to help create the bifurcation diagram and pendulum visualisation. All generated code was thoroughly read, re-commented and tested with extreme values.
- A a dialogue with Gemini was also used to discuss the mathematical content of this project, though this was relatively limited.
- I also often used the AI "autocomplete" feature, on by default in VS Code to complete some lines of code and comments for me.